Київський національний університет імені Тараса Шевченка

Кафедра мережевих і Інтернет технологій

Лабораторна робота №11

з дисципліни

«Технології створення програмних продуктів»

# ШАБЛОН INTERPRETER

Група: МІТ-м11

Студент: Переверзев Олексій

Київ 2026

**Завдання:** Розробіть простий інтерпретатор математичних виразів, який може обробляти операції множення та ділення за допомогою змінних.

Код реалізації:

```
using System;
using System.Collections.Generic;

namespace InterpreterTask
{
    class Context
    {
        Dictionary<string, int> variables;

        public Context()
        {
            variables = new Dictionary<string, int>();
        }

        public int GetVariable(string name)
        {
            return variables[name];
        }

        public void SetVariable(string name, int value)
        {
            if (variables.ContainsKey(name))
                variables[name] = value;
            else
                variables.Add(name, value);
        }
    }

    interface IExpression
    {
        int Interpret(Context context);
    }

    class NumberExpression : IExpression
    {
        string name;

        public NumberExpression(string variableName)
        {
            name = variableName;
        }

        public int Interpret(Context context)
        {
            return context.GetVariable(name);
        }
    }

    class MultiplyExpression : IExpression
    {
        IExpression leftExpression;
        IExpression rightExpression;

        public MultiplyExpression(IExpression left, IExpression right)
        {
            leftExpression = left;
            rightExpression = right;
        }

        public int Interpret(Context context)
        {
```

```
            return leftExpression.Interpret(context) *
rightExpression.Interpret(context);
        }
    }

    class DivideExpression : IExpression
    {
        IExpression leftExpression;
        IExpression rightExpression;

        public DivideExpression(IExpression left, IExpression right)
        {
            leftExpression = left;
            rightExpression = right;
        }

        public int Interpret(Context context)
        {
            int rightVal = rightExpression.Interpret(context);

            if (rightVal == 0)
            {
                Console.WriteLine("Помилка: Спроба ділення на нуль!");
                return 0;
            }

            return leftExpression.Interpret(context) / rightVal;
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Шаблон Interpreter (Множення та Ділення) ===\n");

            Context context = new Context();

            context.SetVariable("a", 20);
            context.SetVariable("b", 4);
            context.SetVariable("c", 2);

            Console.WriteLine("Значення змінних: a = 20, b = 4, c = 2\n");

            IExpression multiplyExpr = new MultiplyExpression(new NumberExpression("a"),
new NumberExpression("b"));
            Console.WriteLine($"Результат (a * b): {multiplyExpr.Interpret(context)}");

            IExpression divideExpr = new DivideExpression(multiplyExpr, new
NumberExpression("c"));
            Console.WriteLine($"Результат (a * b) / c:
{divideExpr.Interpret(context)}");

            Console.ReadKey();
        }
    }
}
```

Реалізовано шаблон Interpreter для обчислення математичних виразів. Context: Служить сховищем для змінних та їхніх значень. NumberExpression виступає термінальним символом, який безпосередньо повертає значення з контексту. MultiplyExpression та DivideExpression є нетермінальними виразами, які рекурсивно викликають метод Interpret() для своїх операндів та виконують

відповідну арифметичну дію. Це дозволяє гнучко будувати складні вирази у вигляді дерева об'єктів.

Результат виконання програми:

```
=== Шаблон Interpreter (Множення та Ділення) ===

Значення змінних: a = 20, b = 4, c = 2

Результат (a * b): 80
Результат (a * b) / c: 40
```

**Висновок:** Під час виконання лабораторної роботи було досліджено шаблон Interpreter. Цей патерн довів свою ефективність для вирішення задач, де потрібно інтерпретувати певні послідовності команд або математичні вирази. Головною перевагою є можливість легко розширювати граматику (додавати нові операції), просто створюючи нові класи нетермінальних виразів, не змінюючи при цьому логіку роботи інших компонентів системи.